

Hands-on Lab: Unit Testing



Unit Testing Lab

Estimated time needed: 30 minutes

Objectives

After completing this lab you will be able to:

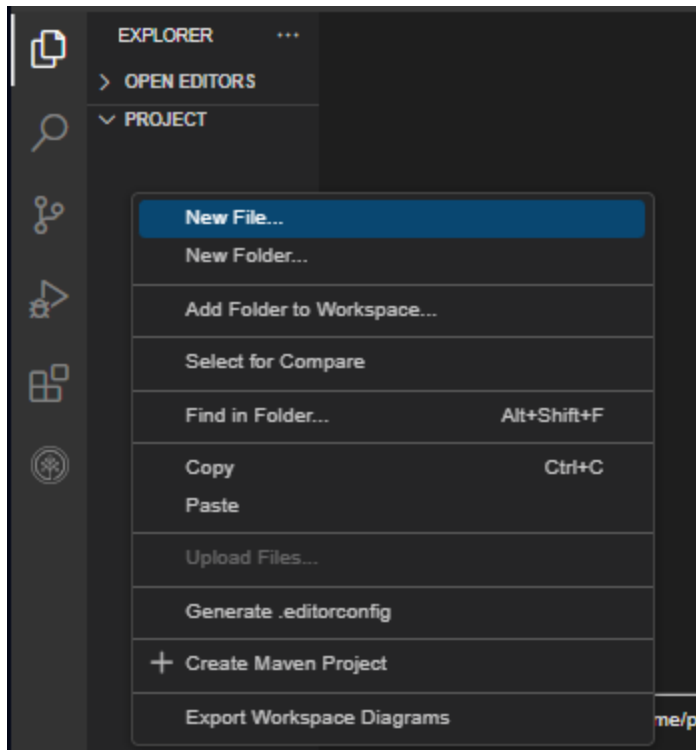
- Write unit tests to test a function.
- Run unit tests and interpret the results.

About the lab environment

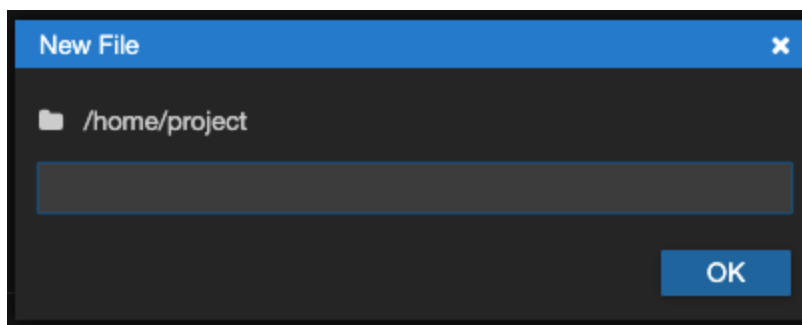
Cloud IDE is an open-source IDE(Integrated Development Environment), that can be run on desktop or on cloud. You will be using the Cloud IDE to do this lab. When you log into the Cloud IDE environment, you are presented with a 'dedicated computer on the cloud' exclusively for you. This is available to you as long as you work on the labs. Once you log off, this 'dedicated computer on the cloud' is deleted along with any files you may have created. So, it is a good idea to finish your labs in a single session. If you finish part of the lab and return to the Theia lab later, you may have to start from the beginning. Plan to work out all your Theia labs when you have the time to finish the complete lab in a single session.

Create a new python file named mymodule.py

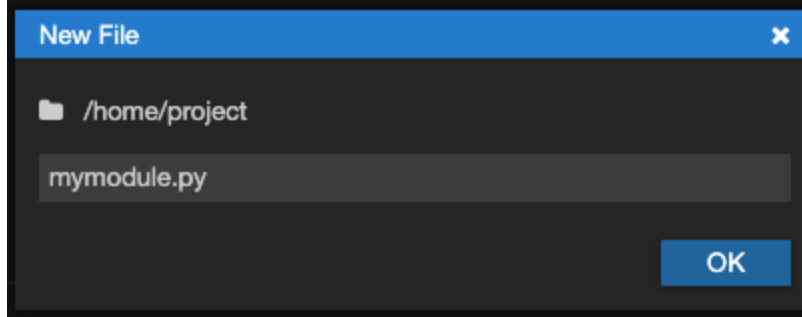
On the window, Right click on the **Explorer** and select **New File** option, as shown in the image below.



A pop up appears with title **New File**, as shown in the image below.



Enter "mymodule.py" as the file name and click **OK**.



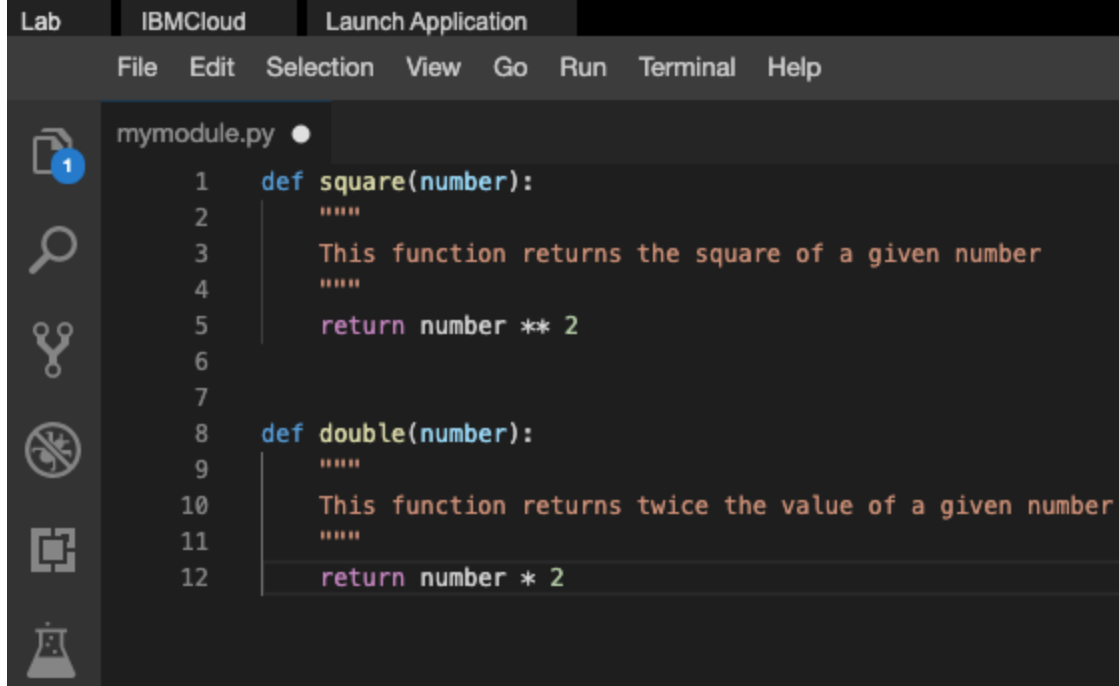
A file "mymodule.py" will be created for you.

You are now ready to add code to mymodule.py

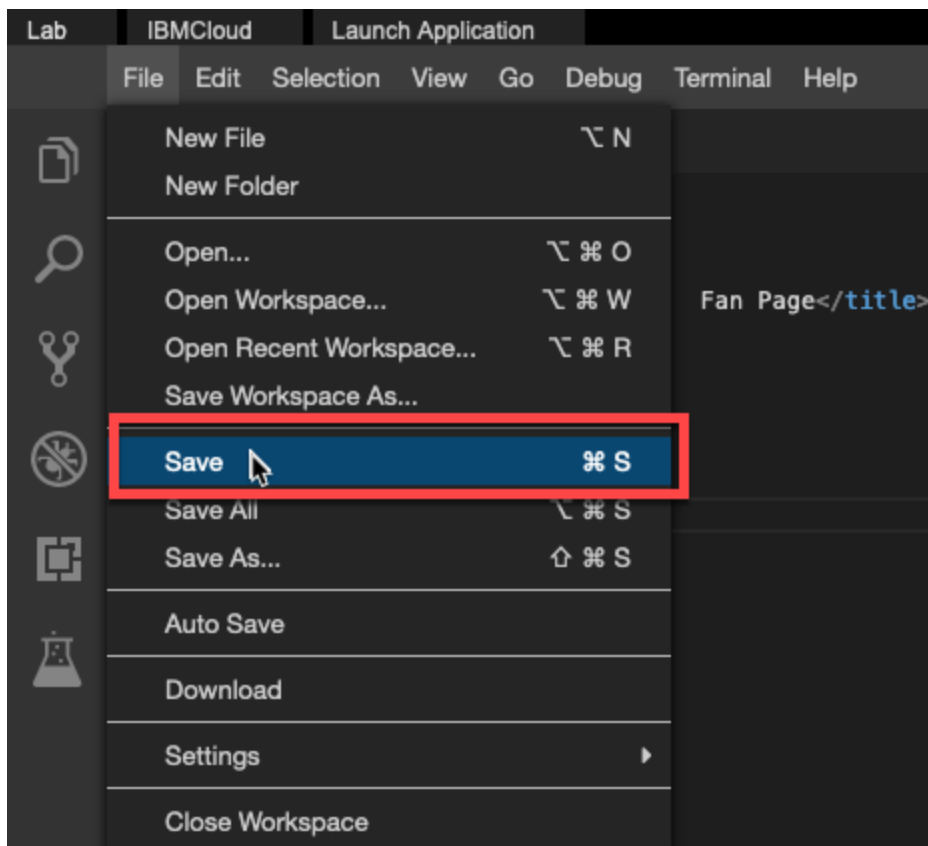
Copy and paste the below code into mymodule.py

```
def square(number):  
    """  
    This function returns the square of a given number  
    """  
    return number ** 2  
  
def double(number):  
    """  
    This function returns twice the value of a given number  
    """  
    return number * 2
```

You should see a screen like this now.



Save the file by using the Save option in the File Menu.



Write Unit Tests

Write the unit tests for square function

Let us write test cases for these three scenarios.

- When 2 is given as input the output must be 4.
- When 3.0 is given as input the output must be 9.0.
- When -3 is given as input the output must not be -9.

Write the unit tests for double function

Let us write test cases for these three scenarios.

- When 2 is given as input the output must be 4.
- When -3.1 is given as input the output must be -6.2.
- When 0 is given as input the output must be 0.

Create a new file and name it as test_mymodule.py

Copy and paste the below code into test_mymodule.py

```
import unittest

from mymodule import square, double

class TestSquare(unittest.TestCase):

    def test1(self):
        self.assertEqual(square(2), 4) # test when 2 is given as input the ou
        self.assertEqual(square(3.0), 9.0) # test when 3.0 is given as input
        self.assertNotEqual(square(-3), 9) # test when -3 is given as input
```

```
class TestDouble(unittest.TestCase):
```

```
    def test1(self):
```

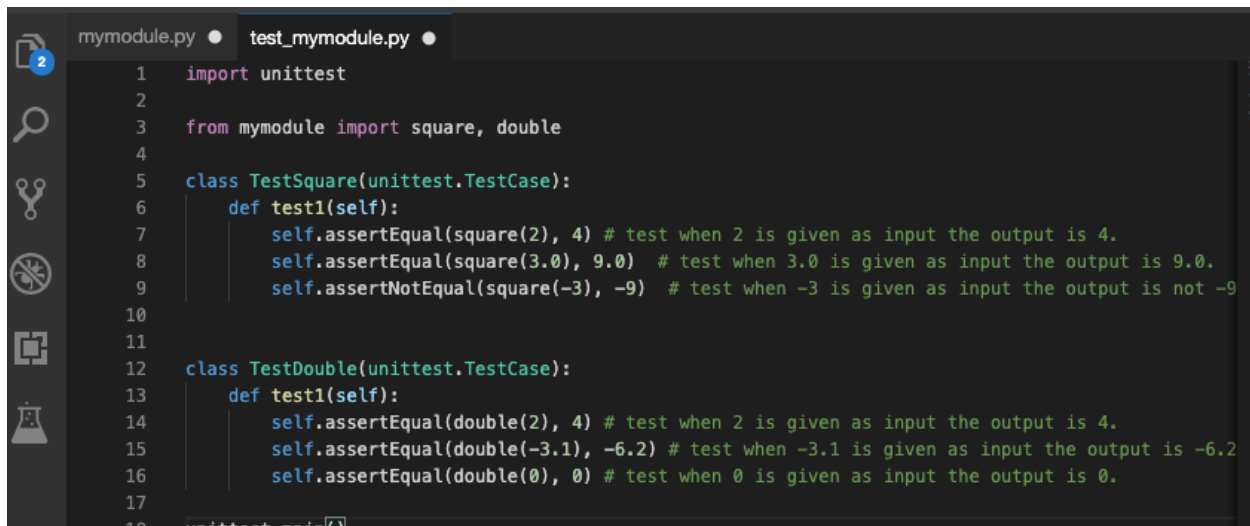
```
        self.assertEqual(double(2), 4) # test when 2 is given as input the ou
```

```
        self.assertEqual(double(-3.1), -6.2) # test when -3.1 is given as inp
```

```
        self.assertEqual(double(0), 0) # test when 0 is given as input the ou
```

```
unittest.main()
```

You should see a screen like this now.



```
1 import unittest
2
3 from mymodule import square, double
4
5 class TestSquare(unittest.TestCase):
6     def test1(self):
7         self.assertEqual(square(2), 4) # test when 2 is given as input the output is 4.
8         self.assertEqual(square(3.0), 9.0) # test when 3.0 is given as input the output is 9.0.
9         self.assertNotEqual(square(-3), -9) # test when -3 is given as input the output is not -9
10
11
12 class TestDouble(unittest.TestCase):
13     def test1(self):
14         self.assertEqual(double(2), 4) # test when 2 is given as input the output is 4.
15         self.assertEqual(double(-3.1), -6.2) # test when -3.1 is given as input the output is -6.2
16         self.assertEqual(double(0), 0) # test when 0 is given as input the output is 0.
17
18 unittest.main()
```

Understanding the code

Imports: We begin by importing the `unittest` module, which provides the tools needed to write and run tests. We also import the `square` and `double` functions from `mymodule` — these are the functions we want to test.

TestSquare class: In `unittest`, related tests are grouped into a class that inherits from `unittest.TestCase`. The `TestSquare` class contains a single test method `test1` — all test methods must start with the word `test` so the test runner can automatically detect and execute them. Inside `test1`, we use `assertEqual` to verify that `square(2)` returns 4 and that `square(3.0)` returns 9.0, confirming the function works correctly for both integer and float inputs. We use `assertNotEqual` to confirm that `square(-3)` does not return -9, since squaring a negative number must always yield a positive result.

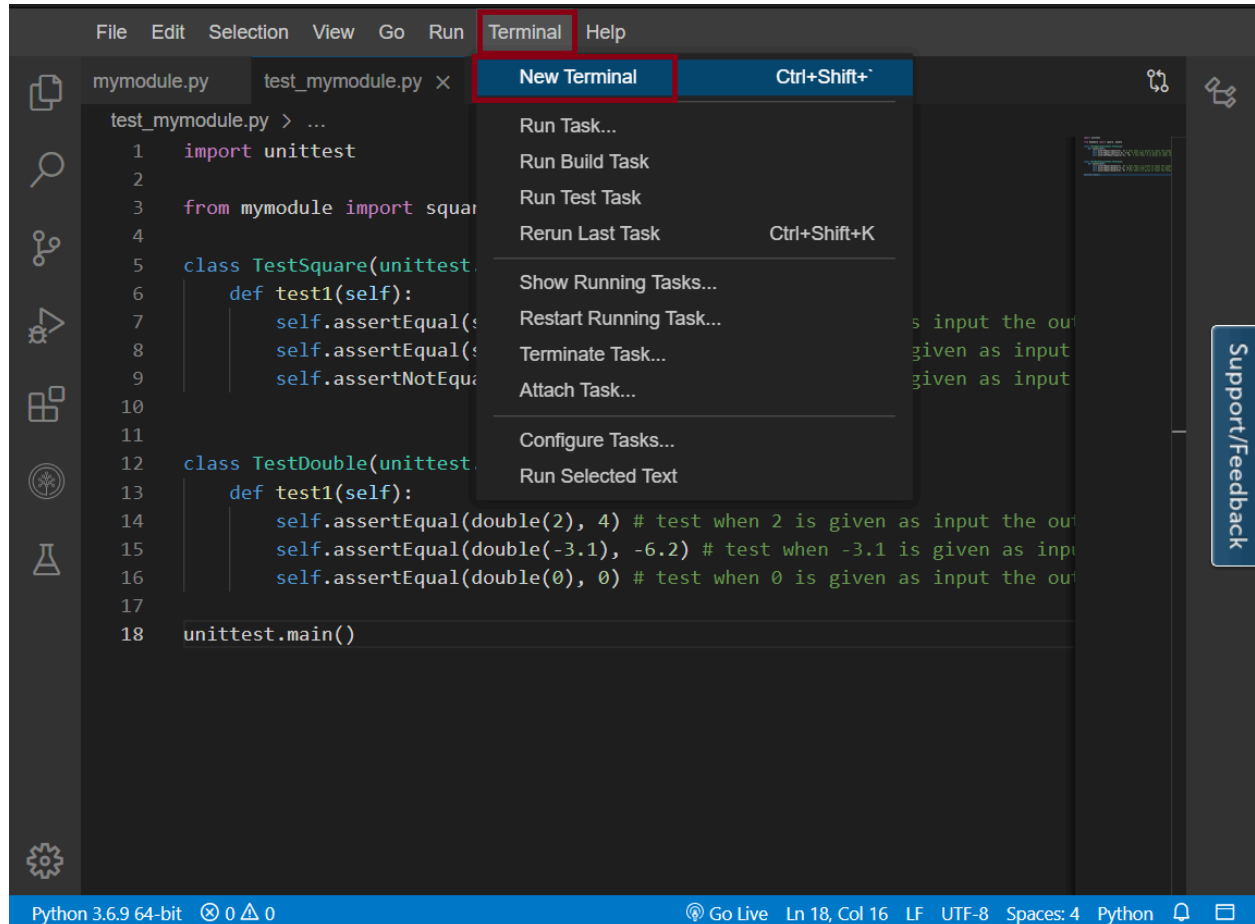
TestDouble class: The `TestDouble` class follows the same structure and tests the

-3.1 returns -6.2, and that doubling 0 returns 0 — this last case is an important edge case to confirm the function behaves correctly at the boundary.

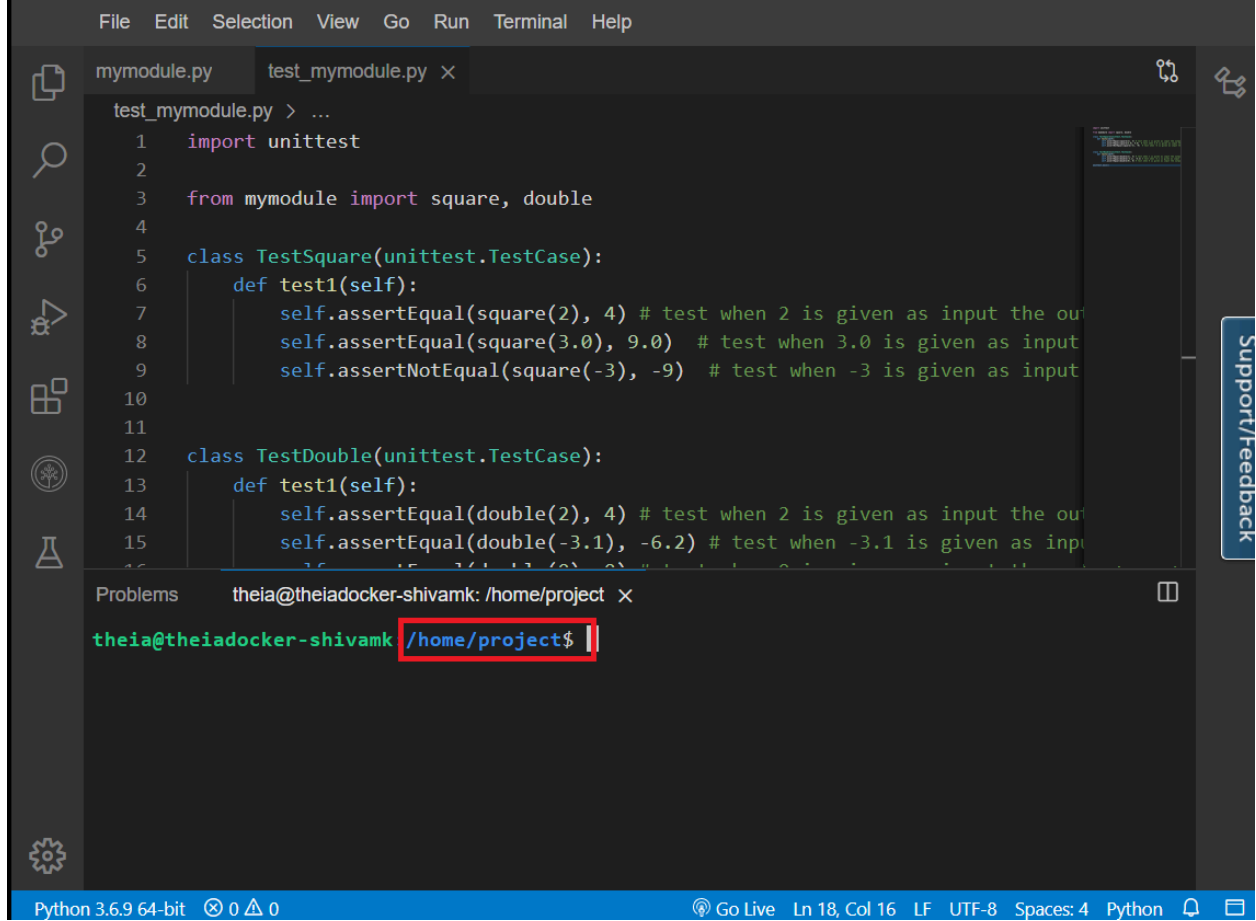
`unittest.main()`: This line at the end tells Python to run all the test cases defined in the module when the script is executed directly. It automatically discovers and runs every method that starts with `test` across all `TestCase` classes in the file.

Run tests

To run tests, click on the "Terminal" and then click on the "New Terminal"



It will open the terminal



The screenshot displays the JupyterLab environment. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The left sidebar contains icons for file explorer, search, source control, console, and a settings gear. The main editor area shows the file `test_mymodule.py` with the following Python code:

```
test_mymodule.py > ...
1  import unittest
2
3  from mymodule import square, double
4
5  class TestSquare(unittest.TestCase):
6      def test1(self):
7          self.assertEqual(square(2), 4) # test when 2 is given as input the out
8          self.assertEqual(square(3.0), 9.0) # test when 3.0 is given as input
9          self.assertNotEqual(square(-3), -9) # test when -3 is given as input
10
11
12  class TestDouble(unittest.TestCase):
13      def test1(self):
14          self.assertEqual(double(2), 4) # test when 2 is given as input the out
15          self.assertEqual(double(-3.1), -6.2) # test when -3.1 is given as input
16
```

Below the editor is a terminal window titled `Problems theia@theiadocker-shivamk: /home/project`. The terminal shows the prompt `theia@theiadocker-shivamk /home/project$` with a red box highlighting the prompt and a cursor. The status bar at the bottom indicates `Python 3.6.9 64-bit`, `0 0 0`, `Go Live`, `Ln 18, Col 16`, `LF`, `UTF-8`, `Spaces: 4`, `Python`, and a notification icon.

Run command `python3 test_mymodule.py` and this will run the tests.

You should see a screen like this now.

The screenshot shows a VS Code editor with two tabs: `mymodule.py` and `test_mymodule.py`. The `test_mymodule.py` file contains the following Python code:

```
test_mymodule.py > ...
1  import unittest
2
3  from mymodule import square, double
4
5  class TestSquare(unittest.TestCase):
6      def test1(self):
7          self.assertEqual(square(2), 4) # test when 2 is given as input the output should be 4
8          self.assertEqual(square(3.0), 9.0) # test when 3.0 is given as input the output should be 9.0
9          self.assertNotEqual(square(-3), -9) # test when -3 is given as input the output should not be -9
10
11
12  class TestDouble(unittest.TestCase):
13      def test1(self):
14          self.assertEqual(double(2), 4) # test when 2 is given as input the output should be 4
15          self.assertEqual(double(-3.1), -6.2) # test when -3.1 is given as input the output should be -6.2
16
```

Below the code editor, the `Problems` panel shows the command `python3 test_mymodule.py` executed in the terminal. The output of the command is:

```
theia@theiadocker-shivamk:/home/project$ python3 test_mymodule.py
..
-----
Ran 2 tests in 0.000s

OK
theia@theiadocker-shivamk:/home/project$
```

The status bar at the bottom indicates the environment is `Python 3.6.9 64-bit` and the file is encoded in `UTF-8`.

An **OK** in the last line indicates that all tests passed successfully.

FAILED in the last line indicates that at least one test has failed, and python prints which test or tests failed.

Write unit tests for the given function

Here is a function that accepts two arguments and returns their sum.

Copy and paste the below code into mymodule.py and then save the file.

```
def add(a,b):  
    """  
    This function returns the sum of the given numbers  
    """  
    return a + b
```

Write test cases for these scenarios.

- When 2 and 4 are given as input the output must be 6.
- When 0 and 0 are given as input the output must be 0.
- When 2.3 and 3.6 are given as input the output must be 5.9.
- When the strings 'hello' and 'world' are given as input the output must be 'helloworld'.
- When 2.3000 and 4.300 are given as input the output must be 6.6.
- When -2 and -2 are given as input the output must **not** be 0. (Hint : Use `assertNotEqual`)

Author(s)

Ramesh Sannareddy

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.

Hands-on Lab: Unit Testing



Unit Testing Lab

Estimated time needed: 30 minutes

Objectives

After completing this lab you will be able to:

- Write unit tests to test a function.
- Run unit tests and interpret the results.

[About the lab environment](#) →